

Canevas EDA — version complète

Basé sur ton canevas original, avec les sections manquantes ajoutées (marquées [AJOUT]). À utiliser comme référence pour chaque projet, pas à réciter par cœur.

1. Compréhension du problème (Business understanding)

À documenter :

- Objectif du projet
- Variable cible (target)
- Type de problème : classification / régression / clustering / séries temporelles
- Questions métiers à répondre

[AJOUT] Contexte et contraintes métier :

- Qui utilise ce modèle et pour quelle décision ? (ex : un associé PE qui filtre des cibles)
- Coût d'une erreur : un faux positif coûte-t-il plus cher qu'un faux négatif, ou l'inverse ?
- Contraintes de déploiement : le modèle doit-il être interprétable (réglementation, confiance métier) ou la performance brute suffit-elle ?

2. Chargement des données

Vérifications :

```
df.head()  
df.shape  
df.columns  
df.info()
```

À commenter : nombre de lignes, nombre de colonnes, types de variables (int, float, object, datetime, bool), mémoire utilisée.

[AJOUT] Vérifications supplémentaires :

- Encodage du fichier et séparateur (utf-8, virgule vs point-virgule) si CSV externe
- `df.sample(10)` en plus de `head()` — `head()` cache parfois des problèmes qui apparaissent plus loin
- Optimisation mémoire des dtypes si le dataset est gros (category pour les variables catégorielles à faible cardinalité)

3. Qualité des données

3.1 Valeurs manquantes

```
df.isnull().sum()  
df.isnull().mean()*100
```

À analyser : colonnes concernées, pourcentage. Stratégie : suppression, imputation moyenne/médiane, imputation par modèle, catégorie « Unknown ».

[AJOUT] Le pattern des valeurs manquantes compte autant que le pourcentage :

- Visualiser avec une matrice de valeurs manquantes (bibliothèque `missingno`) pour voir si l'absence est aléatoire ou corrélée à d'autres variables
- Une valeur manquante n'est parfois pas aléatoire : si elle est corrélée à la target, c'est un signal de fuite de données (voir section [AJOUT] plus bas)

3.2 Doublons

```
df.duplicated().sum()
```

Nombre de doublons, suppression ou conservation justifiée.

3.3 Cohérence des données

- Valeurs négatives impossibles
- Dates incohérentes
- Catégories mal orthographiées
- Valeurs hors domaine métier

4. Analyse univariée

4.1 Variables numériques

```
df.describe().T
```

Métrique	Ce qu'il faut regarder
mean	Tendance centrale
median	Robustesse aux outliers
std	Dispersion
min/max	Valeurs extrêmes
quartiles	Répartition des données

Visualisations obligatoires : histogramme, KDE, boxplot.

```
sns.histplot(df[col], kde=True)
sns.boxplot(x=df[col])
```

À commenter : symétrie/asymétrie, outliers, multimodalité, besoin de transformation (log, Box-Cox).

[AJOUT] Quantifier plutôt que juste observer visuellement :

- `df[col].skew()` et `df[col].kurtosis()` — met un chiffre sur ce que l'œil voit dans l'histogramme
- Test de normalité si pertinent pour le modèle choisi (Shapiro-Wilk pour petits échantillons, Kolmogorov-Smirnov sinon)

4.2 Variables catégorielles

```
df[col].value_counts()
df[col].value_counts(normalize=True)*100
sns.countplot(y=df[col])
```

À commenter : catégories dominantes, catégories rares, déséquilibre éventuel.

[AJOUT] Décision explicite pour les catégories rares : les regrouper en « Autre » si elles représentent moins de ~1-2% des lignes, sinon elles ajoutent du bruit à l'encodage sans apporter de signal.

5. Analyse des outliers

Méthode IQR :

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
borne_inf = Q1 - 1.5*IQR
```

```
borne_sup = Q3 + 1.5*IQR
outliers = df[(df[col] < borne_inf) | (df[col] > borne_sup)]
```

À documenter : nombre d'outliers, pourcentage, décision (conserver / supprimer / capper / transformer).

[AJOUT] L'IQR ne regarde qu'une variable à la fois. Pour des outliers multivariés (une combinaison de valeurs individuellement normales mais ensemble suspectes), utiliser Isolation Forest ou la distance de Mahalanobis — utile quand plusieurs variables sont corrélées entre elles.

6. Analyse bivariée

6.1 Numérique vs numérique

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')

sns.scatterplot(data=df, x='X', y='Y')
```

À commenter : corrélations fortes (> 0.7), corrélations négatives, relations non linéaires, variables redondantes.

[AJOUT] Une corrélation forte entre deux features (pas avec la target) signale de la multicolinéarité, qui pose problème pour les modèles linéaires et l'interprétation des coefficients. Vérifier avec le VIF (Variance Inflation Factor) — un VIF $> 5-10$ signale une variable à retirer ou combiner.

6.2 Catégorielle vs numérique

```
sns.boxplot(data=df, x='categorie', y='valeur')
```

À commenter : différences de médianes, variabilité par groupe, catégories influentes.

[AJOUT] Test ANOVA (ou Kruskal-Wallis si les distributions ne sont pas normales) pour confirmer statistiquement si la différence entre groupes est significative, plutôt que de juger seulement à l'œil sur le boxplot.

6.3 Catégorielle vs catégorielle

```
pd.crosstab(df['A'], df['B'], normalize='index')
sns.heatmap(pd.crosstab(df['A'], df['B']), annot=True)
```

[AJOUT] Le tableau croisé montre la relation mais pas sa force. Ajouter un test du Chi carré ($\chi^2_{contingency}$) et le V de Cramér pour quantifier l'intensité de l'association entre deux variables catégorielles.

7. Analyse de la variable cible

Régression : distribution de la cible, outliers, transformation logarithmique si nécessaire.

Classification :

```
df['target'].value_counts(normalize=True)
```

À commenter : classes équilibrées ou non, risque de biais, métriques adaptées (F1, ROC-AUC, etc.).

[AJOUT] Si les classes sont déséquilibrées (ex : 90/10), documenter dès cette étape la stratégie de correction envisagée : `class_weight='balanced'`, SMOTE/oversampling, ou simplement le choix d'une métrique robuste au déséquilibre (F1, PR-AUC plutôt qu'accuracy). Décider maintenant évite d'improviser au moment de la modélisation.

8. Analyse temporelle (si applicable)

- Format datetime
- Valeurs manquantes dans le temps
- Fréquence des observations

```
df.groupby('date')['target'].mean().plot(figsize=(12,5))
```

À commenter : tendances, saisonnalité, ruptures, anomalies temporelles.

9. [AJOUT] Fuite de données (data leakage) — section dédiée

C'est le point que ton canevas mentionne en une ligne dans les 12 questions, mais qui mérite sa propre vérification systématique, car c'est l'erreur la plus fréquente et la plus coûteuse en EDA.

- Est-ce qu'une variable contient de l'information qui ne serait pas connue au moment de la prédiction réelle ? (ex : une variable calculée après l'événement qu'on essaie de prédire)
- Est-ce que le split train/test a été fait AVANT tout calcul de statistiques (moyenne, normalisation) ? Toute stat calculée sur l'ensemble complet avant le split fuit de l'information du test vers le train
- Pour des données groupées (plusieurs lignes par entreprise/client) : est-ce que le même groupe peut apparaître à la fois dans train et test ?

10. [AJOUT] Pistes de feature engineering

À noter au fil de l'EDA, pas seulement à la fin — souvent les idées viennent en regardant les distributions et corrélations :

- Ratios entre variables corrélées métier (ex : revenu / nombre d'employés)
- Regroupement de catégories rares
- Transformation log sur les variables très asymétriques
- Interaction entre deux variables si le boxplot bivarié suggère un effet combiné

11. Synthèse des observations

Élément	Observation
Valeurs manquantes	Ex : 12 % dans income
Doublons	Ex : 45 supprimés
Outliers	Ex : 3 % sur revenue
Distribution	Ex : asymétrie à droite
Corrélations	Ex : revenue–ebitda = 0.82
Target	Ex : classes déséquilibrées (90/10)

12. Actions de prétraitement décidées — checklist finale

- Suppression des doublons
- Traitement des valeurs manquantes
- Traitement des outliers
- Encodage des variables catégorielles
- Normalisation / standardisation
- Transformation logarithmique

- Sélection de variables
- Création de nouvelles features

[AJOUT]

- Split train/test effectué avant tout calcul de statistique (voir section fuite de données)
- Seed aléatoire fixé et documenté pour la reproductibilité

Structure idéale d'un notebook

- 1. Business Understanding
- 2. Data Loading
- 3. Data Quality Assessment
- 4. Univariate Analysis
- 5. Outlier Analysis
- 6. Bivariate Analysis
- 7. Target Variable Analysis
- 8. Time Series Analysis (optionnel)
- 9. Data Leakage Check [AJOUT]
- 10. Feature Engineering Ideas [AJOUT]
- 11. Key Findings
- 12. Preprocessing Decisions
- 13. Conclusion

Les questions à toujours te poser

Les 12 questions originales, plus 3 ajouts :

- Le dataset est-il complet ?
- Y a-t-il des doublons ?
- Les types de données sont-ils corrects ?
- Y a-t-il des valeurs impossibles ?
- La distribution est-elle normale ou asymétrique ?
- Y a-t-il des outliers importants ?
- Quelles variables sont les plus dispersées ?
- Quelles variables sont fortement corrélées ?
- Existe-t-il des relations non linéaires ?
- La variable cible est-elle équilibrée ?
- Y a-t-il un risque de fuite de données (data leakage) ?
- Quelles transformations seront nécessaires avant la modélisation ?
- [AJOUT] Le split train/test a-t-il été fait avant tout calcul statistique ?
- [AJOUT] Qui utilisera ce modèle et quel est le coût d'une erreur (faux positif vs faux négatif) ?
- [AJOUT] Les valeurs manquantes suivent-elles un pattern, ou sont-elles vraiment aléatoires ?